



Systematyczny Przegląd Literatury

Metody Badawcze w Informatyce

Julia Konarzewska 212038

Kanstantsin Chyzheuski 211984

Elvira Oliinyk 211994

Spis treści

1. Projekt Badawczy.....	3
1.1. Tytuł.....	3
1.2. Opiekun.....	3
1.3. Cele i krótki opis.....	3
2. Plan Systematycznego Przeglądu Literatury	3
2.1. Cele i pytania badawcze	3
2.2. Słowa kluczowe	4
2.3. Search strings	4
2.4. Bazy danych literatury	4
2.5. Kryteria włączenia.....	5
2.6. Kryteria wykluczenia.....	5
2.7. Kryteria jakości.....	5
2.8. Ekstrakcja danych	6
2.9. Proces SLR	6
3. Wyniki Systematycznego Przeglądu Literatury	7
3.1. Wyniki liczbowe	7
3.2. Artykuły zakwalifikowane do ekstrakcji danych.....	8
3.3. Statystyki publikacji	12
3.4. Wstępne dane z ekstrakcji.....	14
4. Wnioski	18
4.1 Proces SLR	19
4.2 Wyniki SLR	19
5. Literatura	20

1. Projekt Badawczy

1.1. Tytuł

Generowanie kodu w pipeline'ach CI/CD na podstawie artefaktów projektowych (CG/CI/CD)

1.2. Opiekun

dr inż. Andrzej Wardziński

1.3. Cele i krótki opis

Celem projektu jest zbadanie możliwości rozszerzenia tradycyjnych pipeline'ów CI/CD poprzez wprowadzenie wcześniejszego etapu automatyzacji, w którym kod źródłowy jest automatycznie generowany na podstawie artefaktów projektowych, takich jak modele UML lub wyniki narzędzi CASE.

Proponowana koncepcja, określana jako CG/CI/CD (Continuous Generation, Integration and Deployment), zakłada integrację automatycznej generacji kodu z pipeline'ami CI/CD (np. z wykorzystaniem narzędzi takich jak Jenkins), co umożliwi w pełni zautomatyzowany proces dostarczania oprogramowania.

2. Plan Systematycznego Przeglądu Literatury

2.1. Cele i pytania badawcze

Celem niniejszego przeglądu literatury (SLR) jest analiza aktualnego stanu badań dotyczących generowania kodu w procesach CI/CD. Przegląd ma na celu zrozumienie, w jaki sposób automatyczne generowanie kodu jest wykorzystywane w nowoczesnych pipeline'ach DevOps oraz jakie rozwiązania i podejścia są obecnie stosowane.

Na podstawie głównego celu sformułowano następujące cele szczegółowe:

- identyfikacja i analiza istniejących badań dotyczących generowania kodu w kontekście CI/CD;
- określenie najczęściej wykorzystywanych narzędzi, technologii i podejść;
- analiza korzyści oraz wyzwań związanych z wykorzystaniem generowania kodu w pipeline'ach CI/CD;
- wskazanie potencjalnych luk badawczych i kierunków dalszych badań.

Aby osiągnąć powyższe cele, sformułowano następujące pytania badawcze:

1. **RQ1:** W jaki sposób generowanie kodu jest wykorzystywane w procesach CI/CD według literatury?
2. **RQ2:** Jakie narzędzia, frameworki i podejścia są stosowane do automatycznego generowania kodu?
3. **RQ3:** Jakie rodzaje artefaktów projektowych (np. modele UML) wykorzystuje się do generowania kodu?

4. **RQ4:** Jakie są główne korzyści wynikające z wykorzystania generowania kodu w CI/CD?

2.2. Słowa kluczowe

W celu przygotowania zapytań wyszukiwania (search strings) wyodrębniono słowa kluczowe na podstawie sformułowanych pytań badawczych. Następnie dla każdego z nich dobrano synonimy oraz alternatywne terminy stosowane w literaturze naukowej.

Generowanie kodu: "code generation", "automatic code generation", "code synthesis", "program synthesis", "code automation".

CI/CD i DevOps: "CI/CD", "continuous integration", "continuous deployment", "continuous delivery", "DevOps".

Kontekst inżynierii oprogramowania: "software development", "software engineering", "DevOps pipeline", "build automation".

Podejścia modelowe: "model-driven", "model driven", "MDE", "UML", "CASE tools", "software models".

2.3. Search strings

Na podstawie wybranych słów kluczowych opracowano zapytania wyszukiwania (search strings). W zależności od bazy danych zastosowano różne warianty zapytań, aby zwiększyć liczbę oraz trafność wyników.

Dodatkowo, w trakcie procesu wyszukiwania każdy z członków zespołu eksperymentalnie testował różne warianty search strings, aby znaleźć najbardziej efektywne zapytania, zapewniające równowagę pomiędzy liczbą wyników a ich trafnością.

Zastosowane search strings w poszczególnych bazach:

IEEE Xplore:

*("code generation" OR "automatic code generation" OR "code synthesis")
AND ("CI/CD" OR "continuous integration" OR "continuous deployment")
AND ("software development" OR "software engineering" OR "DevOps")*

Scopus:

("code generation" OR "program synthesis" OR "code synthesis") AND ("CI/CD" OR "continuous integration" OR "continuous deployment" OR "DevOps") AND ("model-driven" OR "model driven" OR MDE OR UML OR "CASE tools" OR "software models")

SpringerLink:

("model-driven" OR UML) AND ("code generation") AND ("continuous integration" OR "CI/CD")

2.4. Bazy danych literatury

W ramach przeglądu literatury wykorzystano następujące bazy danych naukowych:

- IEEE Xplore
- Scopus

- SpringerLink

2.5. Kryteria włączenia

Do przeglądu literatury włączano artykuły spełniające następujące kryteria:

- publikacje z lat 2020-2026;
- artykuły napisane w języku angielskim;
- publikacje naukowe (artykuły konferencyjne lub czasopisma);
- prace związane z tematyką CI/CD, DevOps lub pipeline'ów;
- artykuły dotyczące generowania kodu lub automatyzacji procesów.

2.6. Kryteria wykluczenia

Z przeglądu wykluczono artykuły, które:

- nie były związane z tematyką CI/CD lub generowania kodu;
- dotyczyły innych dziedzin (np. hardware, systemy wbudowane bez kontekstu CI/CD);
- nie zawierały żadnych konkretnych rozwiązań, metod lub przykładów;
- były duplikatami tych samych publikacji;
- były zbyt ogólne i nie odnosiły się bezpośrednio do tematu badania.

2.7. Kryteria jakości

Po zastosowaniu kryteriów włączenia i wykluczenia wybrane artykuły zostały dodatkowo ocenione pod względem jakości. Celem tej oceny było sprawdzenie, czy publikacje są wystarczająco rzetelne i zawierają wartościowe informacje do dalszej analizy.

Ocena jakości została przeprowadzona na podstawie kilku prostych kryteriów:

1. Czy cel pracy jest jasno określony;
2. Czy opisane metody lub podejścia (np. generowanie kodu) są zrozumiałe i szczegółowe;
3. Czy artykuł zawiera analizę lub eksperymenty;
4. Czy przedstawiono wyniki oraz ich ocenę;
5. Czy opisano wykorzystane dane, narzędzia lub środowisko;
6. Czy wnioski odpowiadają na postawione cele;
7. Czy wyniki zostały porównane z innymi badaniami.

Każde kryterium oceniano w prosty sposób:

1 punkt – jeśli artykuł spełnia dane kryterium,

0 punktów – jeśli nie spełnia.

Następnie obliczano sumę punktów dla każdego artykułu. Maksymalnie artykuł mógł uzyskać 7 punktów.

Przyjęto, że artykuły, które uzyskały co najmniej **4 punkty**, są wystarczająco dobre jakościowo i mogą zostać uwzględnione w dalszej analizie.

Ocena jakości została przeprowadzona przez członków zespołu w ramach przypisanych baz danych, a następnie wyniki zostały wzajemnie zweryfikowane, aby zapewnić większą wiarygodność oceny.

2.8. Ekstrakcja danych

Z wybranych artykułów wyodrębniono najważniejsze informacje, które pozwalają odpowiedzieć na postawione pytania badawcze. Dane te zostały zapisane w formie tabeli – arkusz Excel, co ułatwia ich analizę i porównanie.

Z każdego artykułu zbierano następujące informacje:

- DOI;
- tytuł artykułu;
- autorzy;
- rok publikacji;
- źródło;
- baza;
- link;
- cel pracy;
- zastosowane metody lub podejścia;
- główne wyniki i wnioski;
- związek artykułu z tematem CI/CD oraz generowaniem kodu.

Proces ekstrakcji danych został przeprowadzony przez członków zespołu w ramach przypisanych baz danych, a następnie wyniki zostały wzajemnie sprawdzone, aby zapewnić poprawność i spójność zebranych informacji. W przypadku wątpliwości decyzje były podejmowane wspólnie.

2.9. Proces SLR

Proces przeglądu literatury został przeprowadzony etapami, żeby uporządkować pracę i znaleźć tylko najbardziej pasujące artykuły.

Na początku określono pytania badawcze, a następnie na ich podstawie dobrano słowa kluczowe i przygotowano search stringi. Potem wykonano wyszukiwanie artykułów w wybranych bazach danych.

Po zebraniu wyników przeprowadzono wstępną selekcję na podstawie tytułów i abstraktów. Następnie zastosowano kryteria włączenia i wykluczenia, żeby odrzucić artykuły, które nie pasowały do tematu.

Na końcu wybrano artykuły do dalszej analizy i wykonano wstępną ekstrakcję danych.

Etapy procesu:

1. Określenie pytań badawczych;
2. Wybór słów kluczowych i przygotowanie search stringów;
3. Wyszukiwanie artykułów w bazach danych;
4. Wstępna selekcja na podstawie tytułu i abstraktu;
5. Zastosowanie kryteriów włączenia i wykluczenia;
6. Wybór artykułów do analizy;
7. Wstępna ekstrakcja danych.

Wyszukiwanie przeprowadzono w następujących bazach danych:

- IEEE Xplore

- Scopus
- SpringerLink

Dla każdej bazy użyto przygotowanych wcześniej search stringów.

Praca została podzielona pomiędzy członków zespołu. Każda osoba była odpowiedzialna za wyszukiwanie artykułów w jednej bazie danych, a mianowicie: IEEE Xplore – Elvira Oliinyk; Scopus – Julia Konarzewska; SpringerLink – Kanstantsin Chyzheuski.

Wybrane artykuły oraz wyniki selekcji były wzajemnie sprawdzane przez członków zespołu, aby uniknąć błędów i zwiększyć wiarygodność wyników.

W trakcie realizacji przeglądu wykorzystano:

- arkusz kalkulacyjny (Excel) do zapisywania i porządkowania wyników;
- Zotero do zarządzania literaturą;
- Python do analizy danych oraz tworzenia wykresów i diagramów.

3. Wyniki Systematycznego Przeglądu Literatury

W niniejszym rozdziale przedstawiono wyniki systematycznego przeglądu literatury.

Obejmują one wyniki liczbowe selekcji artykułów, analizę statystyczną oraz wstępne dane uzyskane z ekstrakcji danych z wybranej literatury.

3.1. Wyniki liczbowe

W wyniku wyszukiwania w bazach danych zidentyfikowano łącznie 310 artykułów. Po usunięciu duplikatów pozostały 292 publikacje. Na podstawie analizy tytułów oraz streszczeń, wybrano 32 artykuły. Do pełnej analizy zostało wybranych 21 publikacji. Poniżej zostały przedstawione wyniki liczbowe w podziale na bazy danych.

Liczba znalezionych artykułów:

- IEEE Xplore: 47
- Scopus: 147
- SpringerLink: 116

Liczba artykułów po usunięciu duplikatów:

- IEEE Xplore: 47
- Scopus: 142
- SpringerLink: 103

Liczba artykułów po filtracji na podstawie tytułu oraz abstraktu:

- IEEE Xplore: 11
- Scopus: 12
- SpringerLink: 9

Liczba artykułów po przeprowadzeniu pełnej analizy: 21

- IEEE Xplore: 9

- Scopus: 7
- SpringerLink: 7

3.2. Artykuły zakwalifikowane do ekstrakcji danych

W Tabeli 1 przedstawiono 21 artykułów, które zakwalifikowano do ekstrakcji danych. Opisane publikacje spełniły kryteria włączające, wykluczające oraz uzyskały co najmniej 4 punkty w ocenie jakości.

Tabela 1: Artykuły zakwalifikowane do ekstrakcji danych

ID	DOI	Tytuł	Autorzy	Rok	Źródło
L1	10.1109/ICTBIG68706.2025.11323919	Agentic AI-Driven CI/CD Pipelines for Autonomous Software Delivery	Ravi Nagvekar	2025	2025 IEEE 5th International Conference on ICT in Business Industry & Government (ICTBIG)
L2	10.1109/ICECMSN68058.2025.11382867	AI-Driven DevOps for Intelligent Automation in Continuous Software Delivery Pipelines	Kiran Raj K M, Karthik K Poojary, Abhay, Aishwarya R S, Lathesh Kumar S R	2025	2025 5th International Conference on Evolutionary Computing and Mobile Sustainable Networks (ICECMSN)
L3	10.1145/3691620.3695346	PACGBI: A Pipeline for Automated Code Generation from Backlog Items	Mahja Sarschar, Gefei Zhang, Annika Nowak	2024	2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)
L4	10.1109/EIECS59936.2023.10435475	A Research of Page Automatic Publishing Scheme Based on Low Code Platform	Yan Bian; Qing Zou; Weirui Yue; Kun Huang; Le Liu; Xiao Han	2023	2023 3rd International Conference on Electronic Information Engineering and

					Computer Science (EIECS)
L5	10.1109/MODELS-C68889.2025.00014	A Model-Driven Approach for CI/CD	Hugo Da Gião	2025	2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)
L6	10.1109/ICSCC62041.2024.10690452	ZRB: A Novel Framework for Code Generation and Task Automation	Go Frendi Gunawan; Mukhlis Amien	2024	2024 10th International Conference on Smart Computing and Communication (ICSCC)
L7	10.1109/MODELS67397.2025.00026	A Metamodel for Reengineering CI/CD Pipelines	Hugo da Gião; Vasco Amaral; Gregor Engels; André Flores; Rui Pereira; Stefan Sauer, Jácome Cunha	2025	2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS)
L8	10.1109/ITMS67030.2025.11236630	MDDOAI: A Model-Driven DevOps Approach to CI/CD Automation	Uldis Karlovs-Karlovskis; Oksana Nikiforova; Oscar Pastor; Kristijans Vēveris	2025	2025 66th International Scientific Conference on Information Technology and Management Science of Riga Technical University (ITMS)
L9	10.1109/CLOUD49709.2020.00041	Auto-Generation of Domain-Specific Systems:	Saurabh Sinha; Tara Astigarraga;	2020	2020 IEEE 13th International Conference on

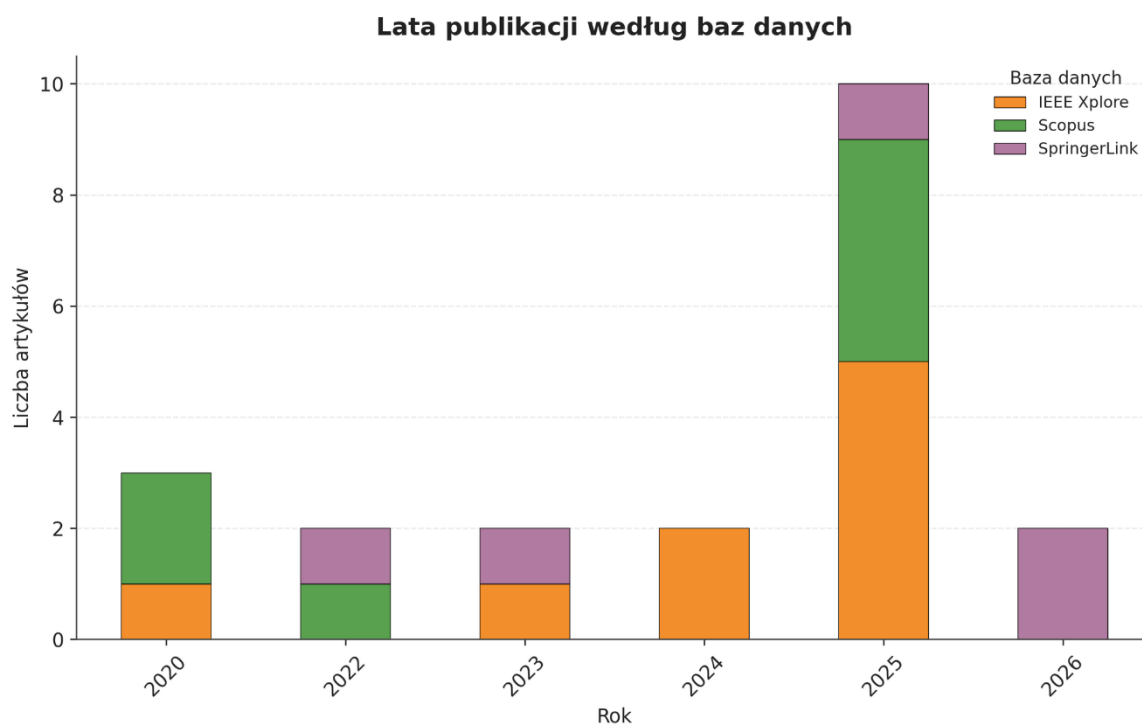
		Cloud-Hosted DevOps for Business Users	Richard B. Hull; Nerla Jean-Louis; Vugranam Sreedhar; Hao Chen		Cloud Computing (CLOUD)
L10	10.1002/sys.21789	Models2Code: Autonomous model-based generation to expedite the engineering process	Paniagua C.; Caso F.L.	2025	Systems Engineering
L11	10.1109/MNE3SD67637.2025.11323237	A Metamodel for Simplifying Infrastructure Deployment Using Vagrant	Somda F.H.; Kabore A.S.; Zerbo B.	2025	2025 IEEE Multi-Conference on Natural and Engineering Sciences for Sahel's Sustainable Development, MNE3SD 2025
L12	10.1145/3417990.3421446	TwinOps - DevOps meets model-based engineering and digital twins for the engineering of CPS	Hugues J.; Hristosov A.; Hudak J.J.; Yankel J.	2020	Proceedings - 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems, MODELS-C 2020 - Companion Proceedings
L13	10.5381/jot.2020.19.2.a16	Continuous deployment of trustworthy smart IoT systems	Ferry N.; Nguyen P.H.; Song H.; Rios E.; Iturbe E.; Martinez S.; Rego A.	2020	Journal of Object Technology

L14	10.5220/0013422800003932	ADA-Gen: Iterative and Incremental Generation of Full-Stack Apps for Learning Agile/DevOps Software Development Practices	Duong T.N.B.	2025	International Conference on Computer Supported Education, CSEDU - Proceedings
L15	10.1145/3550356.3563134	CPSAML: A Language and Code Generation Framework for Digital Twin based Monitoring of Mobile Cyber-Physical Systems	Fend A.; Bork D.	2022	Proceedings - ACM/IEEE 25th International Conference on Model Driven Engineering Languages and Systems, MODELS 2022: Companion Proceedings
L16	10.14279/eceasst.v84.2680	Towards X-as-Models in the Context of CI/CD	Teumert S.; Tegeler T.	2025	Electronic Communications of the EASST
L17	10.1186/s13677-025-00761-w	ArtifactOps and ArtifactDL: a methodology and a language for conceptualizing and operationalising different types of pipelines	Raúl MiñónJosu Diaz-de-ArcayaAna I. Torre-BastidaJuan López-de-ArmentiaGorka ZarateLander BonillaAsier Garcia-PerezJon Aguirre-Usandizaga	2025	Journal of Cloud Computing

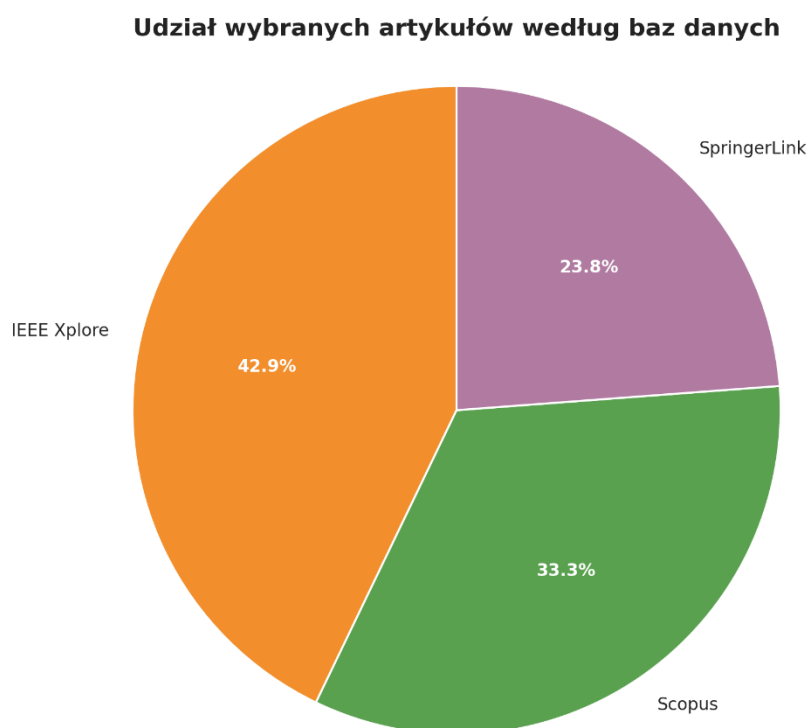
L18	10.1007/s10270-022-01003-2	Using DevOps toolchains in Agile model-driven engineering	Jörn Guy SüßSamantha SwiftEban Escott	2022	Software and Systems Modeling
L19	10.1007/978-3-031-36847-9_4	Continuous Alignment Between Software Architecture Design and Development in CI/CD Pipelines	Davide TaibiYuanfang CaiIngo WeberMehdi MirakhorliMic hael W. GodfreyJohn T. StoughPatrizio Pelliccione	2023	Software Architecture
L20	10.1007/978-981-95-4724-1_15	Automated UML Generation: A Framework for Class Diagram Synthesis and Multimodal Validation	Van-Viet NguyenHuu-Khanh NguyenKim-Son NguyenMinh-Hue Luong ThiThe-Vinh NguyenDuc-Quang Vu	2026	Future Data and Security Engineering
L21	10.1007/978-3-032-12092-2_10	Development of a Model-Driven DevOps Solution Based on Context-Engineered LLM Code Generation: PROFES Doctoral Symposium	Uldis Karlovs-Karlovskis	2026	Product-Focused Software Process Improvement. Industry, Doctoral-Symposium, Tutorial, and Workshop Papers

3.3. Statystyki publikacji

Na rysunku 1 pokazano wykres przedstawiający, w których latach zostały opublikowane wybrane artykuły z podziałem na wykorzystane bazy danych. Najwięcej publikacji ukazało się w 2025 roku, co sugeruje rosnące zainteresowanie badaną tematyką.



Rysunek 1: Rozkład czasowy publikacji w latach 2020-2026 z podziałem na wykorzystane bazy danych



Rysunek 2: Udział wybranych artykułów według baz danych wyrażony w procentach

Na rysunku 2 przedstawiono wykres pokazujący udział wybranych artykułów według baz danych. Artykuły pochodzą głównie z bazy IEEE Xplore, jednakże nie jest to przeważająca liczba, dzięki czemu został zachowany balans.

3.4. Wstępne dane z ekstrakcji

Po zakończeniu etapu wyboru artykułów, rozpoczęto proces ekstrakcji danych. Z analizowanych publikacji wyodrębniono kluczowe informacje dotyczące zastosowanych metod oraz wyników badań. Pozyskane dane zostały przedstawione w Tabeli 2.

Tabela 2: Wstępne dane z ekstrakcji

ID	Cel pracy	Zastosowane metody lub podejścia	Główne wyniki i wnioski
L1	Zaproponowanie frameworku Agentic AI dla w pełni autonomicznych potoków CI/CD w procesie wytwarzania oprogramowania.	Systemy Agentic AI z autonomicznymi agentami, pętlami sprzężenia zwrotnego, procesami samouczenia oraz interakcją w języku naturalnym.	Proponowany framework umożliwia kompleksową automatyzację procesu wytwarzania oprogramowania, w tym generowanie kodu, testowanie oraz wdrażanie przy minimalnej ingerencji człowieka.
L2	Analiza wpływu sztucznej inteligencji na DevOps oraz potoki CI/CD w kontekście inteligentnej automatyzacji dostarczania oprogramowania.	Wykorzystanie uczenia maszynowego, analityki predykcyjnej, głębokiego uczenia, uczenia ze wzmocnieniem oraz dużych modeli językowych (LLM).	Sztuczna inteligencja poprawia automatyzację, stabilność oraz efektywność potoków CI/CD, jednak wyzwaniami pozostają wyjaśnialność modeli oraz optymalizacja zasobów.
L3	Przedstawienie potoku do automatycznego generowania kodu na podstawie elementów backlogu z wykorzystaniem modeli LLM zintegrowanych z CI/CD.	Potok oparty na LLM zintegrowany z GitLab CI, który odczytuje elementy backlogu i generuje kod wraz z automatycznymi merge requestami.	Rozwiązanie automatyzuje proces tworzenia oprogramowania, redukuje pracę manualną i koszty oraz umożliwia równoległy rozwój.
L4	Opracowanie rozwiązania do automatycznej publikacji stron w oparciu o platformy low-code.	Zarządzanie przepływem pracy oparte na szablonach oraz automatyzacja z wykorzystaniem platform low-code.	Rozwiązanie zwiększa efektywność publikowania, zmniejsza liczbę błędów oraz upraszcza przepływ pracy w procesie tworzenia oprogramowania.

L5	Zaproponowanie podejścia opartego na modelach (model-driven) do projektowania i zarządzania potokami CI/CD.	Inżynieria oparta na modelach (model-driven engineering), tworzenie metamodeli oraz automatyczne generowanie konfiguracji CI/CD.	Podjęcie redukuje złożoność i nakład pracy manualnej poprzez umożliwienie automatycznego generowania konfiguracji CI/CD oraz ułatwienie migracji narzędzi.
L6	Wprowadzenie frameworku do generowania kodu oraz automatyzacji zadań w procesie wytwarzania oprogramowania.	Framework oparty na przepływie pracy z wykorzystaniem koncepcji DAG, z zadaniami i automatyzacją zintegrowanymi z narzędziami takimi jak GitHub Actions.	Framework zwiększa produktywność, ogranicza pracę manualną oraz umożliwia efektywną realizację zadań i automatyzację.
L7	Zaproponowanie metamodelu wspierającego migrację oraz reengineering potoków CI/CD.	Inżynieria oparta na modelach (MDE) oraz metamodelowanie potoków CI/CD.	Podjęcie umożliwia automatyczne generowanie równoważnych potoków oraz redukuje błędy podczas migracji.
L8	Zaproponowanie podejścia DevOps opartego na modelach do automatycznego generowania potoków CI/CD na podstawie modeli architektury oprogramowania.	Inżynieria oparta na modelach z wykorzystaniem transformacji ATL oraz narzędzia Acceleo do automatycznego generowania kodu.	Podjęcie umożliwia automatyczne generowanie konfiguracji potoków CI/CD oraz zmniejsza lukę między projektowaniem a wdrożeniem.
L9	Zaproponowanie podejścia DevOps umożliwiającego użytkownikom biznesowym generowanie i zarządzanie systemami bez konieczności posiadania zaawansowanej wiedzy technicznej.	Modelowanie dziedzinowe oraz podejście DevOps z metodologią „specify-check-generate-deploy”.	Podjęcie umożliwia użytkownikom automatyczne generowanie i wdrażanie systemów, zwiększając elastyczność oraz zmniejszając zależność od programistów.
L10	Automatyzacja przejścia od fazy projektu (design) do implementacji w celu redukcji kosztów i czasu inżynierskiego	Wtyczki do narzędzi Eclipse i Papyrus wykorzystujące transformację modeli SysML, biblioteka Velocity do generowania kodu na podstawie szablonów	Framework pozwala na autonomiczną generację szkieletów systemów, skryptów bazodanowych SQL i dokumentacji. Wprowadza moduł walidacji, który aktualizuje model

			SysML na podstawie zmian wprowadzonych ręcznie w kodzie.
L11	Przedstawienie frameworka opartego na metamodelu, który zastępuje ręcznie pisane pliki Vagrantfile walidowanymi modelami EMF	Model-Driven Engineering, Eclipse Modeling Framework (EMF), Acceleo do transformacji model-tekst	Podejście zredukowało rozmiar konfiguracji o około 50%, skróciło czas tworzenia o 75% i niemal wyeliminowało błędy składniowe (spadek z 5-7 do 0-1 błędów).
L12	Unifikacja Model-Based Engineering, Digital Twins oraz praktyk DevOps w celu usprawnienia procesu inżynierskiego systemów cyberfizycznych (Cyber-Physical Systems)	TwinOps, DevOps, automatyczne generowanie kodu oraz konteneryzacja	TwinOps przyspiesza zbieranie danych i umożliwia ich analizę w celu aktualizacji wymagań systemowych.
L13	Wsparcie Continuous Deployment (CD) godnych zaufania systemów IoT w rozproszonych infrastrukturach	Model-Driven Engineering, silnik models@run.time, język DSL do specyfikacji wdrożeń, ThingML do generowania kodu	Przedstawiony język GENESIS umożliwia Continuous Deployment i orkiestrację Inteligentnych Systemów IoT dzięki wykorzystaniu technik i metod model-driven.
L14	Nauka praktyk Agile/DevOps poprzez generowanie w pełni funkcjonalnych aplikacji full-stack.	Integracja platformy Jira, repozytorium Bitbucket oraz modeli LLM (GPT-4o).	Narzędzie dostarczyło okazji do nauki pisania user stories, podziału zadań i Continuous Integration.
L15	Przedstawienie frameworka CPSAML do modelowego rozwoju mobilnych systemów cyberfizycznych (Cyber-Physical Systems)	Transformacja modeli w języku DSL do SysML 2, automatyczne generowanie kodu za pomocą Xtend i Java	Wykazano, że podejście to oszczędza znaczną ilość wysiłku implementacyjnego (wygenerowano ponad 9000 linii kodu w studium przypadku) i pozwala na dynamiczne dodawanie/usuwanie komponentów w czasie rzeczywistym.

L16	Zaproponowanie paradygmatu "X-as-Models" jako alternatywy dla tekstowego "X-as-Code" w celu uproszczenia konfiguracji CI/CD	Graficzny język DSL i środowisko Rig do wizualnego projektowania przepływów CI/CD	Modelowanie graficzne potencjalnie eliminuje błędy składniowe YAML i obniża próg wejścia dla osób niebędących ekspertami. Pozwala na walidację logiki potoku przed jego uruchomieniem.
L17	Opracowanie ujednoliconej metodologii (ArtifactOps) oraz języka (ArtifactDL) do projektowania i zarządzania pipeline'ami integrującymi różne typy artefaktów (kod, dane, modele, infrastruktura).	ArtifactOps, ArtifactDL (DSL), DevOps, DataOps, MLOps, IaC, pipeline engineering	Zaproponowano metodologię ArtifactOps oraz język ArtifactDL, które umożliwiają spójne modelowanie i zarządzanie pipeline'ami obejmującymi różne artefakty. Podejście poprawia integrację, automatyzację i zarządzanie cyklem życia pipeline'ów.
L18	Celem artykułu jest pokazanie, jak zastosować narzędzia DevOps w model-driven engineering (MDE), aby zwiększyć zwinność (Agile) procesu wytwarzania oprogramowania.	DevOps toolchains, CI/CD pipelines, Agile, model-driven engineering (MDE), Eclipse/EMF, automatyzacja buildów	Wykazano, że integracja DevOps z MDE jest możliwa i pozwala na automatyzację procesów oraz szybsze wdrożenia, jednak obecne narzędzia MDE nie są w pełni dostosowane do pracy w pipeline'ach CI/CD, co ogranicza ich praktyczne zastosowanie.
L19	Celem artykułu jest zapewnienie ciągłej zgodności między projektowaną architekturą systemu a jego implementacją w procesie CI/CD.	CI/CD pipelines, architecture recovery, continuous alignment, analiza architektury, wykrywanie degradacji architektury	Wykazano, że w procesie CI/CD często dochodzi do rozbieżności między architekturą a kodem. Zaproponowano wykorzystanie pipeline'ów CI/CD oraz metod analizy architektury do ciągłego monitorowania, wykrywania degradacji i utrzymania spójności systemu.
L20	Celem artykułu jest opracowanie podejścia do automatycznego	LLM, PlantUML, pipeline dwumodelowy, multimodal	Autorzy zaproponowali pipeline, w którym jeden model generuje specyfikację

	generowania diagramów klas UML na podstawie specyfikacji technicznych.	validation, vision-language models	techniczną, a drugi tworzy kod PlantUML diagramu klas. Dodatkowo zastosowano walidację multimodalną, aby sprawdzić zgodność diagramu ze specyfikacją.
L21	Celem artykułu jest opracowanie podejścia do automatycznego generowania pipeline'ów CI/CD na podstawie modeli architektury systemu z wykorzystaniem LLM.	Model-Driven Engineering, LLM, model transformation (ATL), code generation (Acceleo), CI/CD pipeline generation	Zaproponowano podejście MDDOAI umożliwiające automatyczne generowanie pipeline'ów CI/CD z modeli architektury. Wykazano, że integracja transformacji modeli i generowania kodu z użyciem LLM pozwala na automatyzację oraz poprawę utrzymania pipeline'ów.

Powyższe dane pozwoliły na posortowanie publikacji według 3 różnych kryteriów:

1. Technologie i podejścia:
 - LLM: L1, L2, L3, L14, L20, L21
 - MDE: L5, L7, L8, L10, L11, L13, L15, L18
 - Low-Code: L4, L16
 - Pipeline Orchestration: L6, L12, L17, L19
2. Dane wejściowe:
 - Modele wizualne: L5, L7, L8, L10, L11, L20
 - Języki specyficzne dla dziedziny: L13, L15, L16, L17
 - Wymagania i tekst: L3, L14
 - Dane Operacyjne: L12, L19
3. Efektywność i rezultaty:
 - Metryki ilościowe: L1, L11, L15
 - Poprawa jakości: L7, L11, L16, L19
 - Bariery: L2, L18

4. Wnioski

W niniejszym rozdziale przedstawiono wnioski wynikające z częściowego przeprowadzenia Systematycznego Przeglądu Literatury. Obejmuje on podsumowanie procesu SLR oraz omówienie uzyskanych wyników.

4.1 Proces SLR

Proces Systematycznego Przeglądu Literatury został przeprowadzony przez członków zespołu z wykorzystaniem różnych zapytań wyszukiwawczych w trzech bazach danych: SpringerLink, Scopus oraz IEEE Xplore. Na etapie wyszukiwania każdy z członków zespołu pracował niezależnie, natomiast wybór artykułów został następnie zweryfikowany wspólnie, zgodnie z przyjętymi kryteriami.

Takie podejście pozwoliło zwiększyć zakres przeszukiwanej literatury, jednak mogło prowadzić do potencjalnych duplikatów.

W celu identyfikacji duplikatów opracowano pipeline w języku Python, który automatycznie porównywał rekordy pochodzące z różnych baz danych. Proces deduplikacji został przeprowadzony na etapie surowych danych, obejmujących łącznie 310 rekordów.

W wyniku tego procesu usunięto 18 duplikatów, uzyskując 292 unikalne rekordy.

Na dalszych etapach selekcji, po zastosowaniu kryteriów włączenia i wykluczenia oraz weryfikacji zespołowej, wybrano 21 artykułów do szczegółowej analizy. Ostateczny zbiór nie zawierał duplikatów, co potwierdza skuteczność zastosowanego podejścia oraz komplementarność wykorzystanych strategii wyszukiwania.

Proces selekcji artykułów, oparty na zdefiniowanych kryteriach oraz wzajemnej weryfikacji członków zespołu, pozwolił na wyodrębnienie prac najbardziej związanych z tematyką CI/CD oraz automatycznej generacji kodu.

4.2 Wyniki SLR

Na podstawie przeprowadzonej analizy można sformułować następujące wnioski:

- Łącznie przeanalizowano 21 artykułów, z czego wszystkie były unikalne
- Największa liczba publikacji pochodzi z bazy IEEE Xplore (9), następnie Scopus (7) oraz SpringerLink (5)
- Najczęściej badane podejścia opierają się na model-driven engineering oraz wykorzystaniu LLM
- Integracja automatycznej generacji kodu z pipeline'ami CI/CD nadal nie jest standardem i pozostaje obszarem badań

Dodatkowo zauważono, że:

- Badania koncentrują się głównie na generacji kodu lub pipeline'ów osobno
- Niewiele prac łączy oba podejścia w jeden spójny proces

- Pojawia się rosnący trend wykorzystania sztucznej inteligencji w automatyzacji procesów DevOps

Wyniki wskazują na istotny potencjał rozwoju koncepcji CG/CI/CD jako rozszerzenia tradycyjnych pipeline'ów.

5. Literatura

Bian, Yan, Qing Zou, Weirui Yue, Kun Huang, Le Liu, i Xiao Han. „A Research of Page Automatic Publishing Scheme Based on Low Code Platform”. *2023 3rd International Conference on Electronic Information Engineering and Computer Science (EIECS)*, września 2023, 568–73. <https://doi.org/10.1109/EIECS59936.2023.10435475>.

Da Gião, Hugo. „A Model-Driven Approach for CI/CD”. *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, października 2025, 52–58. <https://doi.org/10.1109/MODELS-C68889.2025.00014>.

Fend, Andreas, i Dominik Bork. „CPS_{AML}: A Language and Code Generation Framework for Digital Twin Based Monitoring of Mobile Cyber-Physical Systems”. *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*, 23 października 2022, 649–58. <https://doi.org/10.1145/3550356.3563134>.

Ferry, Nicolas, Phu H. Nguyen, Hui Song, et al. „Continuous Deployment of Trustworthy Smart IoT Systems.” *The Journal of Object Technology* 19, nr 2 (2020): 16:1. <https://doi.org/10.5381/jot.2020.19.2.a16>.

Gião, Hugo da, Vasco Amaral, Gregor Engels, et al. „A Metamodel for Reengineering CI/CD Pipelines”. *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, października 2025, 221–31. <https://doi.org/10.1109/MODELS67397.2025.00026>.

Gunawan, Go Frendi, i Mukhlis Amien. „ZRB: A Novel Framework for Code Generation and Task Automation”. *2024 10th International Conference on Smart Computing and Communication (ICSCC)*, lipca 2024, 123–27. <https://doi.org/10.1109/ICSCC62041.2024.10690452>.

Hugues, Jerome, Anton Hristosov, John J. Hudak, i Joe Yankel. „TwinOps - DevOps Meets Model-Based Engineering and Digital Twins for the Engineering of CPS”. *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*, 16 października 2020, 1–5. <https://doi.org/10.1145/3417990.3421446>.

K M, Kiran Raj, Karthik K. Poojary, Abhay, Aishwarya R S, i Lathesh Kumar S R. „AI-Driven DevOps for Intelligent Automation in Continuous Software Delivery Pipelines”. *2025 5th International Conference on Evolutionary Computing and Mobile Sustainable Networks (ICECMSN)*, listopada 2025, 433–40. <https://doi.org/10.1109/ICECMSN68058.2025.11382867>.

Karlovs-Karlovskis, Uldis. „Development of a Model-Driven DevOps Solution Based on Context-Engineered LLM Code Generation: PROFES Doctoral Symposium”. W *Product-Focused Software Process Improvement. Industry, Doctoral-Symposium, Tutorial, and Workshop Papers*, zredagowane przez Giuseppe Scanniello, Valentina Lenarduzzi, Simone Romano, Sira Vegas, i Rita Francese. Springer Nature Switzerland, 2026.
https://doi.org/10.1007/978-3-032-12092-2_10.

Karlovs-Karlovskis, Uldis, Oksana Nikiforova, Oscar Pastor, i Kristijans Vēveris. „MDDOAI: A Model-Driven DevOps Approach to CI/CD Automation”. *2025 66th International Scientific Conference on Information Technology and Management Science of Riga Technical University (ITMS)*, października 2025, 1–9.
<https://doi.org/10.1109/ITMS67030.2025.11236630>.

Miñón, Raúl, Josu Diaz-de-Arcaya, Ana I. Torre-Bastida, et al. „ArtifactOps and ArtifactDL: A Methodology and a Language for Conceptualizing and Operationalising Different Types of Pipelines”. *Journal of Cloud Computing* 14, nr 1 (2025): 42. <https://doi.org/10.1186/s13677-025-00761-w>.

Nagvekar, Ravi. „Agentic AI-Driven CI/CD Pipelines for Autonomous Software Delivery”. *2025 IEEE 5th International Conference on ICT in Business Industry & Government (ICTBIG)*, grudnia 2025, 1–4. <https://doi.org/10.1109/ICTBIG68706.2025.11323919>.

Nguyen, Van-Viet, Huu-Khanh Nguyen, Kim-Son Nguyen, Minh-Hue Luong Thi, The-Vinh Nguyen, i Duc-Quang Vu. „Automated UML Generation: A Framework for Class Diagram Synthesis and Multimodal Validation”. W *Future Data and Security Engineering*, zredagowane przez Tran Khanh Dang, Josef Küng, i Tai M. Chung. Springer Nature, 2026.
https://doi.org/10.1007/978-981-95-4724-1_15.

Paniagua, Cristina, i Fernando Labra Caso. „Models2Code: Autonomous Model-based Generation to Expedite the Engineering Process”. *Systems Engineering* 28, nr 2 (2025): 224–37. <https://doi.org/10.1002/sys.21789>.

Sarschar, Mahja, Gefei Zhang, i Annika Nowak. „PACGBI: A Pipeline for Automated Code Generation from Backlog Items”. *2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, października 2024, 2338–41.
<https://ieeexplore.ieee.org/document/10764968>.

Sinha, Saurabh, Tara Astigarraga, Richard B. Hull, et al. „Auto-Generation of Domain-Specific Systems: Cloud-Hosted DevOps for Business Users”. *2020 IEEE 13th International Conference on Cloud Computing (CLOUD)*, października 2020, 219–28.
<https://doi.org/10.1109/CLOUD49709.2020.00041>.

Somda, Flavien Hervé, Arnold Stéphane Kabore, i Boureima Zerbo. „A Metamodel for Simplifying Infrastructure Deployment Using Vagrant”. *2025 IEEE Multi-Conference on Natural and Engineering Sciences for Sahel's Sustainable Development (MNE3SD)*, 27 listopada 2025, 1–7. <https://doi.org/10.1109/MNE3SD67637.2025.11323237>.

Süß, Jörn Guy, Samantha Swift, i Eban Escott. „Using DevOps Toolchains in Agile Model-Driven Engineering”. *Software and Systems Modeling* 21, nr 4 (2022): 1495–510.
<https://doi.org/10.1007/s10270-022-01003-2>.

Ta, Duong. „ADA-Gen: Iterative and Incremental Generation of Full-Stack Apps for Learning Agile/DevOps Software Development Practices”: *Proceedings of the 17th International Conference on Computer Supported Education*, 2025, 363–70.
<https://doi.org/10.5220/0013422800003932>.

Taibi, Davide, Yuanfang Cai, Ingo Weber, et al. „Continuous Alignment Between Software Architecture Design and Development in CI/CD Pipelines”. W *Software Architecture: Research Roadmaps from the Community*, zredagowane przez Patrizio Pelliccione, Rick Kazman, Ingo Weber, i Anna Liu. Springer Nature Switzerland, 2023.
https://doi.org/10.1007/978-3-031-36847-9_4.

Teumert, Sebastian, i Tim Tegeler. „Towards X-as-Models in the Context of CI/CD”. *Electronic Communications of the EASST* 84, nr Electronic Communications of the EASST, Vol. 84 (2025): 12th International Symposium on Leveraging Applications of Formal Methods, Verification and Validation / 2nd AISoLA-Doctoral Symposium, 2024 (2025).
<https://doi.org/10.14279/ECEASST.V84.2680>.